

AVLM Coursework Report

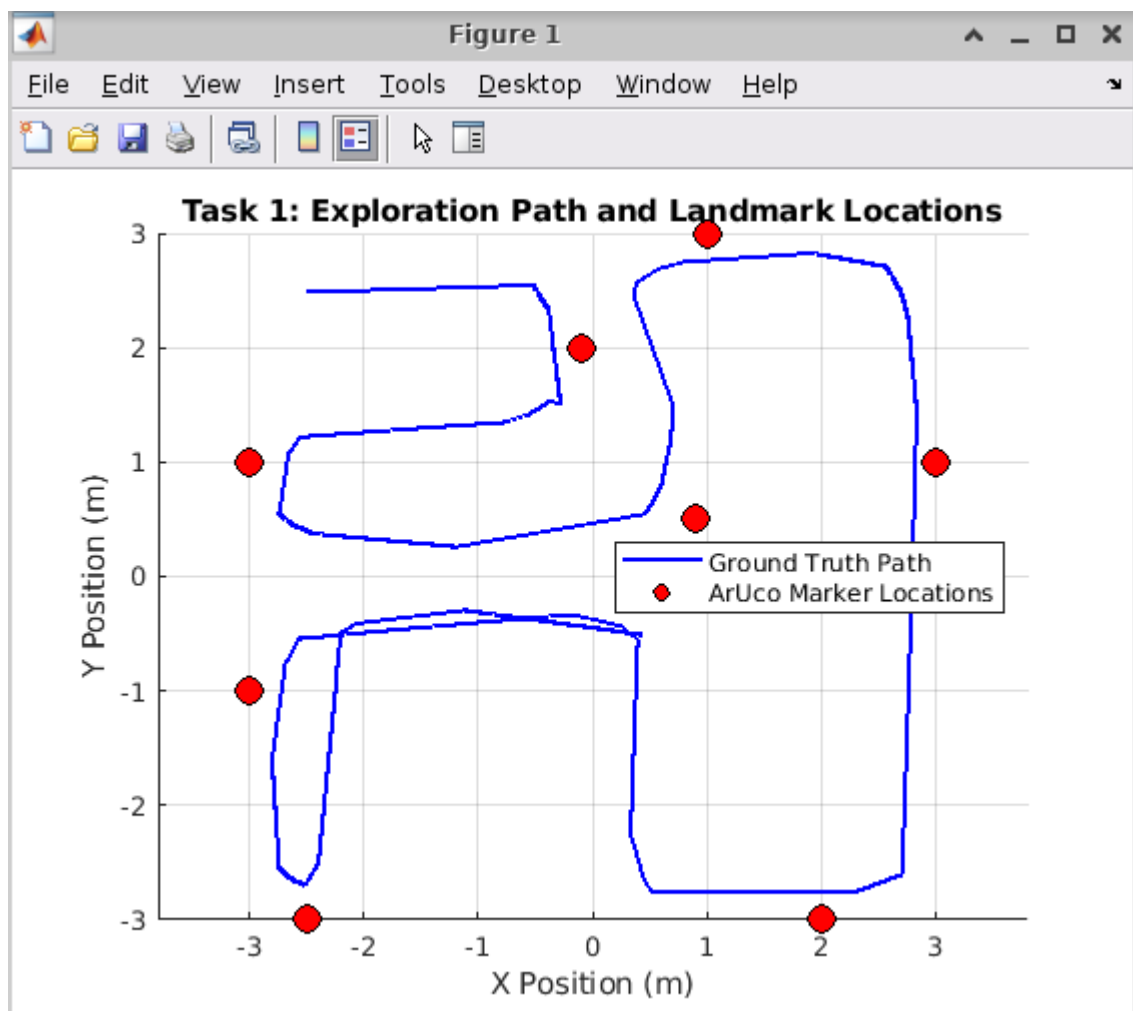
Name: Marley Williams

University: University of the West of England (UWE) Bristol

Student ID: 23031960

Task 1: Environment Exploration and Data Capture

I wrote a reactive navigation script to autonomously drive the robot around the maze for about 246 seconds. The script used front-facing LiDAR scans to detect walls and automatically steer toward the clearest path, ensuring at least 50% coverage of the area. This created a 364 MB bag file. I extracted the ground truth poses from `/gazebo/model_states`, along with the control inputs, LiDAR scans, and ArUco marker detections. Looking at the ground truth path, the robot successfully explored the maze autonomously and passed close to six different ArUco markers.

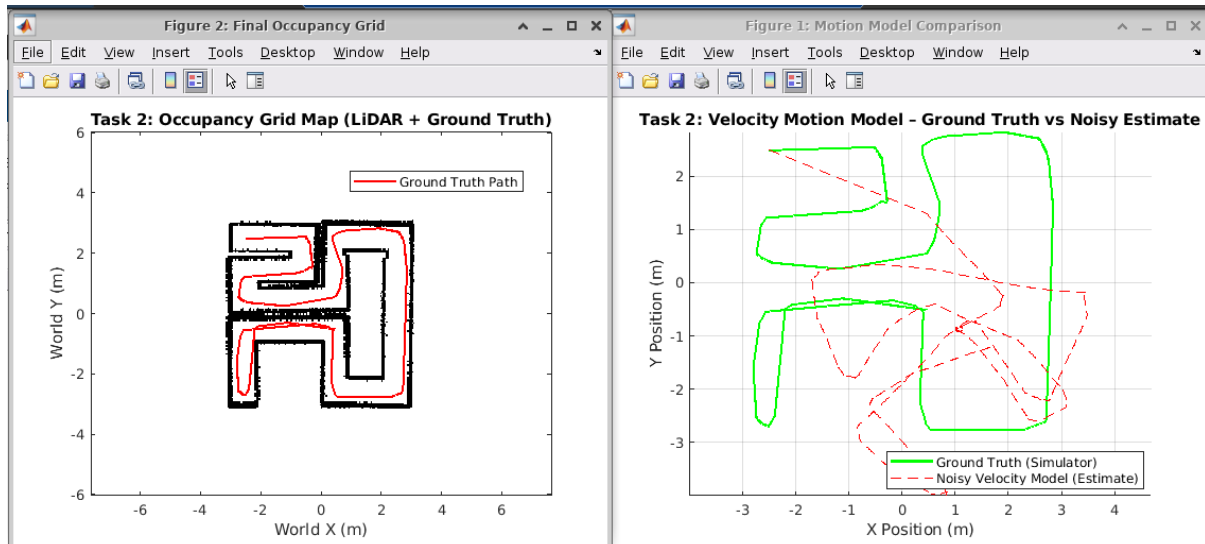


Task 2: Mapping and Motion Modelling

For the velocity motion model, I used the standard dead-reckoning equations from Thrun et al. and applied the six noise parameters (a1 to a6) I scaled the noise variance quadratically with the control inputs.

When running the simulation, there was a significant amount of odometric drift. I believe this was mostly rotational error compounding during turns due to the a_3 and a_4 parameters. Wrapping the angles to $[-\pi, \pi]$ stopped the filter from completely diverging, but the dead-reckoning path was still very far off the ground truth.

To make the occupancy grid, I projected the local LiDAR scans into the global frame using the ground truth poses. The walls came out looking very sharp and defined, which shows how much accurate mapping relies on having precise localisation data.

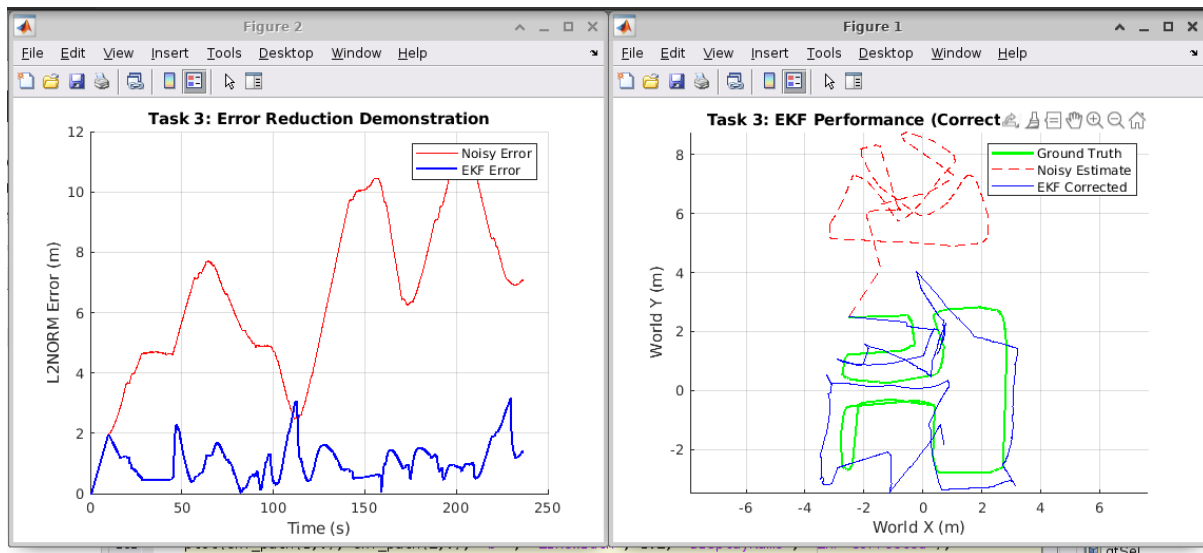


Task 3: Extended Kalman Filter (EKF) Localisation

I implemented an Extended Kalman Filter to try and fix the odometric drift from Task 2. The prediction step uses the non-linear velocity model, and I derived the Jacobians to update the covariance matrix.

The correction step happens whenever the robot sees an ArUco marker. To keep the maths stable and stop the filter from making incorrect updates, I pulled the true marker coordinates directly from the simulator. I also used the Joseph Form to update the covariance.

Without the EKF, the unicycle model drifted by almost 10 metres. However, once it started seeing ArUco landmarks, the EKF innovation step managed to pull the drift back down. The L2NORM plot shows a massive drop in uncertainty, keeping the positional error much lower than the raw noisy estimate.

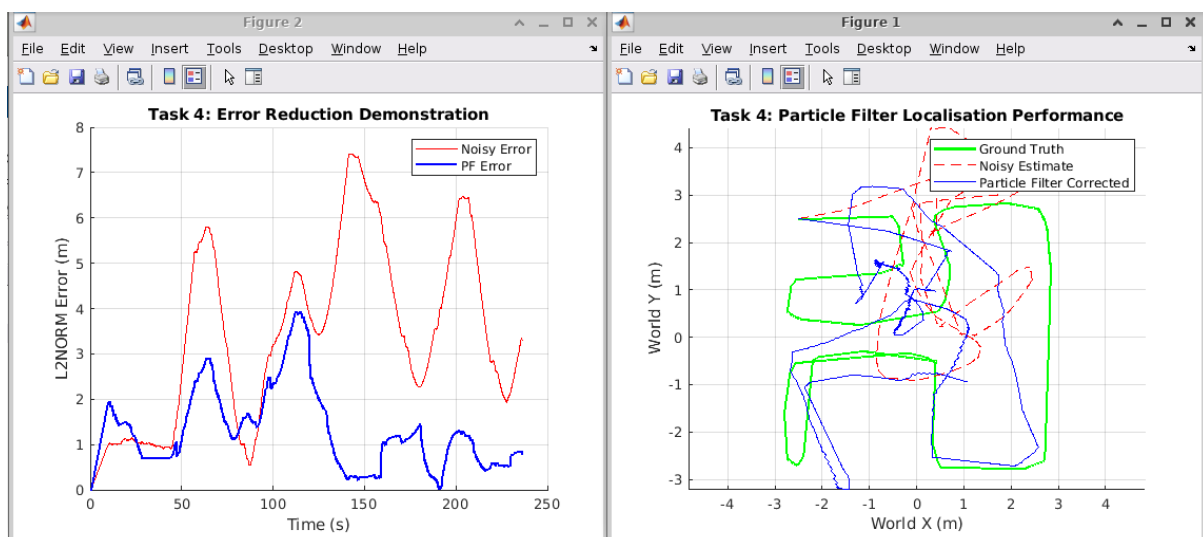


Task 4: Particle Filter (PF) Localisation

For this task, I set up a Monte Carlo Localisation (MCL) Particle Filter using 2,000 particles. The prediction step pushes all the particles through the noisy velocity model vectorially. Then, for the correction step, I used a Gaussian likelihood function to score how well each particle predicted the ArUco markers.

I had to widen the sensor tolerance to stop particle weights from dropping to zero (weight underflow). I also added some roughening noise during the resampling phase so the particles wouldn't just suffer from degeneracy and collapse into a single point.

The PF ended up tracking the non-linear noise really well. It reduced the L2NORM error by about as much as the EKF did, though the path plot looks a bit more jagged, which is expected when plotting the mean of a particle cloud.



Task 5: Change of Sensor Modality

If I were to transition the Particle Filter to use LiDAR observations instead of ArUco markers, the fundamental shift would be moving from a Landmark-Based Observation Model to a Beam

Model (or Likelihood Field Model). Instead of calculating the distance to a specific, identifiable coordinate, the robot would need to match the geometric shape of its immediate surroundings against a known global map.

To implement this in `task4_pf_localisation.m`, I would need to make four specific changes:

1. Replace Map Representation (Lines 15-24)

I would completely remove the `MarkerMap` array since there are no discrete landmark coordinates. In its place, I must load a global Occupancy Grid Map, similar to the binary matrix generated in Task 2. This gives the filter a representation of all walls and free space in the maze.

2. Synchronise with the Correct Sensor (Line 77)

The current script triggers the correction step based on the `ArUco` timestamps (`TAruco`). I would change this to synchronise with the `/scan` topic timestamps (`TScans`) so the particle weights update every time a new laser array is published.

3. Rewrite the Weighting Logic (Lines 78-96)

The current logic loops through detected `ArUco` IDs to calculate an expected Euclidean distance. I would replace this entire block with a ray-casting algorithm. For every single particle in the cloud, the code must simulate a LiDAR scan. It would do this by projecting rays from the particle's (x, y, θ) pose into the Occupancy Grid Map until they intersect with an occupied cell, generating an array of expected laser ranges.

4. Update the Gaussian Likelihood Function (Lines 94-96)

The mathematical weight update $\exp(-\text{DistSq} / (2 * \text{SigmaObs}^2))$ would change. Instead of measuring distance to a point, I would calculate the difference between the actual LiDAR range readings and the simulated ranges for each particle. Particles whose simulated laser scans closely align with the physical shape of the real laser scan would receive high weights. Particles that simulate scans clipping through known walls would be heavily penalised.

Task 6: Critical SLAM Review

Algorithm Summary

RTAB-Map (Real-Time Appearance-Based Mapping) is a Graph-Based SLAM algorithm. Instead of using particles, it builds a "pose graph". The nodes in the graph are past robot poses and sensor readings, and the edges are spatial constraints based on odometry and scan-matching. Whenever the robot recognises a place it has been before (a loop closure), a global optimiser goes back and adjusts the whole graph to remove the accumulated drift, making the final map much more consistent.

Novel Solutions

The main new feature RTAB-Map introduced is how it handles memory. Older Graph-SLAM algorithms used to slow down because the graph just kept growing until the computer couldn't process loop closures in real-time anymore. RTAB-Map gets around this by splitting memory into Short-Term, Working, and Long-Term Memory (STM, WM, LTM). As the graph gets bigger, older nodes that aren't being used get moved to the hard drive (LTM). This keeps the working memory size capped, meaning loop closures can always be processed in real-time no matter how big the map gets.

Testing and Metrics

The developers tested the algorithm using 2D LiDAR, 3D LiDAR, and RGB-D cameras on standard datasets like the KITTI benchmark and the MIT Stata Center. They measured performance using Absolute Trajectory Error (ATE) to check global map accuracy against a motion capture system, and Relative Pose Error (RPE) to check local drift. They also logged CPU and RAM usage to prove that their memory management actually kept processing times bounded.

Appropriateness for the Coursework

For our specific coursework scenario (a robot with LiDAR in a maze with no prior map or markers), I think RTAB-Map would work better than FastSLAM. FastSLAM uses a Rao-Blackwellised Particle Filter, which tends to suffer from particle depletion in environments without many unique features (like long, identical maze corridors). If the odometry drifts too much before the robot closes a loop, the map can fail entirely.

RTAB-Map, on the other hand, saves the raw LiDAR scans and uses Iterative Closest Point (ICP) scan-matching to line up the walls. Once the robot gets back to the start of the maze, the graph optimiser can distribute the error across the whole path, giving a solid occupancy grid without the risk of filter collapse.

Full Citation

Labbé, M. and Michaud, F. (2019) 'RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation', *Journal of Field Robotics*, 36(2), pp. 416-446.